OS 2025 Review

名词解释

1. Context Switch (上下文切换)

上下文切换是操作系统执行的一项基本操作。当操作系统需要将中央处理器（CPU）的使用权从一个进程转移到另一个进程时，就会进行上下文切换。这个过程包括两个主要步骤：首先，操作系统保存当前正在运行进程的所有状态，例如程序计数器、寄存器值等信息。然后，它加载下一个即将执行进程的状态。通过频繁地进行上下文切换，操作系统可以在单个CPU上实现多任务处理，给用户一种多个程序同时运行的错觉。不过，上下文切换本身需要消耗CPU时间，因此它是一项有性能开销的操作。

2. Address Space (地址空间)

地址空间是操作系统为每个进程提供的一个重要抽象。它是一个进程可以用来寻址内存的虚拟地址范围。每个进程都拥有自己独立的、私有的地址空间，这意味着一个进程不能直接访问另一个进程的内存，这提供了很好的隔离和保护。一个进程的地址空间通常包含几个主要部分：代码段，用于存放程序指令；栈，用于存放函数调用信息和局部变量；堆，用于程序运行时动态分配内存。操作系统通过页表等机制，将这个虚拟的地址空间映射到计算机的物理内存上。

3. Translation Lookside Buffer (TLB)

TLB（转译后备缓冲器）是CPU内存管理单元（MMU）内部的一个硬件缓存。它的主要功能是加速虚拟地址到物理地址的转换过程。当CPU生成一个虚拟地址时，硬件会首先检查TLB中是否存在对应的转换记录。如果记录存在（称为TLB命中），地址转换就能立刻完成，速度非常快。如果记录不存在（称为TLB未命中），硬件就需要查询内存中的页表来找到转换关系，这个过程相对慢很多。TLB的存在极大地提升了现代虚拟内存系统的性能。

4. Multi-level Feedback Queue (MLFQ)

多级反馈队列是一种CPU调度算法。它试图同时满足两个相互冲突的目标：为交互式任务提供快速的响应时间，以及为计算密集型任务提供较好的周转时间。为了实现这一点，该算法维护了多个具有不同优先级的任务队列。新到达的任务通常被放入最高优先级的队列。如果一个任务用完了它的时间片，它就会被移动到更低优先级的队列。如果一个任务在时间片用完前主动放弃CPU（例如进行I/O操作），它可能会保持在当前优先级或被提升到更高优先级。

5. Demand Paging (请求分页)

请求分页是实现虚拟内存的一种常见技术。使用这种技术时，操作系统并不会在程序启动时就将整个程序加载到物理内存中。相反，它只在进程实际需要访问某个内存页时，才将该页从磁盘加载到内存。当进程试图访问一个尚未加载的页时，会触发一个名为“页错误”的硬件异常。操作系统会捕获这个异常，从磁盘上找到对应的页，将其加载到物理内存的一个空闲页框中，然后更新页表，最后重新执行导致错误的指令。

6. Critical Section (临界区)

临界区是指程序中一段访问共享资源的代码。这个共享资源可以是一个全局变量、一个数据结构或一个硬件设备。在多线程环境中，如果多个线程同时执行同一个临界区代码，就可能导致数据不一致或其他不可预料的错误，这种情况称为竞争条件（Race Condition）。为了保证程序的正确性，必须确保在任何时刻最多只有一个线程能够进入临界区执行。这种保证被称为互斥（Mutual Exclusion）。

7. Mutex (互斥锁)

互斥锁是一种基本的同步原语，它的主要目的是保护临界区，从而实现互斥。一个互斥锁具有两种状态：锁定和未锁定。它提供两个基本操作：`lock()` 和 `unlock()`。当一个线程想要进入临界区时，它必须首先调用`lock()`来尝试获取锁。如果锁当前是未锁定状态，该线程就能成功获取锁并进入临界区。如果锁已经被其他线程持有，该线程就会被阻塞，直到锁被释放。当线程离开临界区时，它必须调用`unlock()`来释放锁，以便其他等待的线程可以获取它。

8. Condition Variable (条件变量)

条件变量是另一种重要的同步原语，它允许线程在某个特定条件未满足时进入休眠状态，从而避免忙等待。它总是和一个互斥锁配合使用。条件变量提供了两个核心操作：`wait()` 和 `signal()`。一个线程在检查某个条件不满足后，可以调用`wait()`操作。这个操作会自动释放它持有的互斥锁，并让线程进入等待状态。另一个线程在改变了某个条件后，可以调用`signal()`操作来唤醒一个正在等待的线程。

9. Semaphore (信号量)

信号量是一个整数变量，它被用作一种通用的同步工具。它支持两种原子操作：`wait()`（也称为P操作或`sem_wait`）和 `post()`（也称为V操作或`sem_post`）。`wait()`操作会尝试减少信号量的值。如果信号量的值因此变为负数，那么执行`wait()`的线程就会被阻塞。`post()`操作会增加信号量的值，如果此时有线程因该信号量而被阻塞，那么其中一个线程会被唤醒。信号量既可以用于实现互斥（当初始值为1时），也可以用于控制对一组有限资源的访问（当初始值为N时）。

10. Deadlock (死锁)

死锁是指在多任务系统中，两个或多个线程（或进程）相互等待对方持有的资源，导致所有参与者都无法继续执行的一种状态。例如，线程A持有资源1并等待资源2，而线程B持有资源2并等待资源1。在这种情况下，两个线程都会永远等待下去。死锁的发生通常需要满足四个必要条件：互斥、持有并等待、非抢占和循环等待。

11. Inode (索引节点)

索引节点是类Unix文件系统（如ext4）中用于存储文件元数据的核心数据结构。除了文件名之外，一个文件所有的信息都保存在它的索引节点中。这些信息包括文件的大小、所有者、用户组、权限模式、时间戳以及指向存储文件内容的实际数据块的指针。每个文件都有一个唯一的索引节点号，而目录文件则建立了文件名到索引节点号的映射关系。

12. Journaling File System (日志文件系统)

日志文件系统是一种能够提高文件系统可靠性的技术。它的主要目标是在发生系统崩溃（如突然断电）后，能快速恢复文件系统的一致性。在将数据的修改正式写入磁盘的最终位置之前，文件系统会先将这次操作的描述信息写入到一个称为“日志”的特殊区域。如果在写入过程中发生崩溃，系统重启后可以通过读取日志来重做未完成的操作或撤销只完成一半的操作，从而确保文件系统不会处于一个损坏的、不一致的状态。

13. RAID-5 (磁盘阵列-5)

RAID-5是磁盘冗余阵列的一种级别。它将数据以条带化的方式分布在多个磁盘上，以提高读写性能。同时，它也通过存储奇偶校验信息来提供数据冗余。与RAID-4将所有奇偶校验信息存在一个专用磁盘上不同，RAID-5将奇偶校验块均匀地分布在阵列中的所有磁盘上。这种分布式的奇偶校验方式避免了在写操作时单个校验盘成为性能瓶颈的问题。RAID-5可以容忍阵列中任意一块磁盘的损坏而数据不丢失。

14. File Descriptor (文件描述符)

在类Unix操作系统中，文件描述符是一个小的非负整数。它是一个进程用来标识和访问一个已打开文件的句柄。当一个进程成功打开一个文件或创建一个新的I/O通道（如管道）时，内核会在该进程的文件描述符表中创建一个条目，并返回该条目在表中的索引。这个索引就是文件描述符。之后，进程在执行`read()`、`write()`等I/O操作时，就会使用这个文件描述符来指定要操作的对象。

15. Log-structured File System (日志结构文件系统)

日志结构文件系统是一种文件系统设计。它的主要思想是将所有的写入操作都转换成对日志的追加操作。LFS会把所有待写入的数据，包括文件数据和元数据（如inode），先缓存在内存中。然后，它将这些缓存的修改作为一个大的、连续的段一次性地顺序写入到磁盘的空闲空间中。这种方法将大量小的、随机的写操作转换成大的、顺序的写操作，极大地提高了磁盘的写入性能。不过，LFS需要一个后台的清理进程来整理磁盘空间，回收包含旧数据的段。

简答题

1. 死锁的四个必要条件

死锁的发生必须同时满足以下四个条件：

- **互斥（Mutual Exclusion）**：资源是独占性的，在任何时刻一个资源只能被一个进程使用。
- **持有并等待（Hold and Wait）**：一个进程至少持有一个资源，并且正在等待获取其他进程持有的资源。
- **非抢占（No Preemption）**：资源不能被强制地从持有它的进程中抢占。只有持有资源的进程自愿释放后，该资源才能被其他进程使用。
- **循环等待（Circular Wait）**：存在一个进程集合{P0, P1, ..., Pn}，其中P0正在等待P1持有的资源，P1正在等待P2持有的资源，...，而Pn正在等待P0持有的资源，形成一个等待环路。

2. 地址转换过程（TLB命中/未命中）

当CPU需要访问一个虚拟地址时，硬件会执行以下步骤：

1. CPU将虚拟地址发送给内存管理单元（MMU）。
2. MMU首先在TLB（转译后备缓冲器）中查找该虚拟地址对应的页号。
3. **TLB命中**：如果在TLB中找到了有效的转换条目，MMU会立即从中获取物理页号，并与地址中的偏移量组合成最终的物理地址。这个过程非常快。
4. **TLB未命中**：如果TLB中没有找到匹配的条目，硬件（或操作系统）需要访问内存中的页表。它会根据页表基址寄存器和虚拟页号计算出对应页表项（PTE）在内存中的位置，并读取该PTE以获取物理页号。获取到物理页号后，MMU会将其与偏移量组合成物理地址。同时，这个新的转换关系通常会被存入TLB，以便未来的访问能够命中。

3. 为何使用多级页表

使用多级页表主要是为了解决线性页表在稀疏地址空间下占用过多内存的问题。

- **线性页表的问题**：一个简单的线性页表需要为进程虚拟地址空间中的每一个页都创建一个页表项（PTE）。对于一个32位系统，这可能意味着每个进程都需要一个高达4MB的页表。即使进程只使用了很小一部分地址空间，整个页表也必须完整地存在于内存中，造成了巨大的空间浪费。

- **多级页表的解决方案**：多级页表将页表组织成树状结构（例如，页目录和页表）。只有当一个地址空间区域被使用时，系统才需要为该区域分配次一级的页表。如果一个大的地址范围完全未使用，那么在顶层页目录中对应的条目可以为空，从而避免了为这个范围分配任何次级页表的开销。这样，页表所占用的空间就和进程实际使用的内存大小成正比，而不是和虚拟地址空间的大小成正比。

4. SJF与RR算法对性能指标的影响

SJF（最短作业优先）和RR（轮转）对周转时间和响应时间有不同的优化效果。

- **SJF**：该算法总是选择估计运行时间最短的作业来执行。它的主要优点是能够实现最优的平均**周转时间**。因为短作业被优先处理并快速完成，从而降低了所有作业的平均等待时间。但它的缺点是可能导致长作业的响应时间很差，甚至产生饥饿现象。
- **RR**：该算法为每个进程分配一个固定的时间片，并轮流执行。它的主要优点是能够提供很好的平均**响应时间**。通过公平地给每个进程一点运行时间，它确保了所有交互式进程都能很快得到响应。但由于频繁的上下文切换以及未能优先处理短作业，其平均周转时间通常比SJF要差。

5. SSTF算法的问题与SCAN算法的改进

SSTF（最短寻道时间优先）算法在磁盘调度中存在的主要问题是**饥饿（Starvation）**。

- **SSTF的问题**：SSTF总是选择离当前磁头位置最近的磁道请求进行服务。如果此时不断有靠近当前磁头位置的新请求到来，那么离得较远的磁道请求可能永远得不到服务，这就是饥饿现象。
- **SCAN的改进**：SCAN（电梯）算法通过系统地扫描来解决这个问题。磁头会从磁盘的一端移动到另一端，服务沿途的所有请求。到达一端后，它会反向移动，继续服务沿途的请求。这种方式保证了磁头最终会扫过所有磁道，因此任何一个请求的等待时间都有一个确定的上限，从而避免了饥饿问题。

6. 银行家算法

银行家算法是一种用于**避免死锁**的著名算法。它不预防死锁条件的发生，而是在资源分配时通过决策来确保系统不会进入不安全状态，从而避免死锁。

- **基本思想**：该算法要求每个进程在运行前声明其可能需要的各类资源的最大数量。当一个进程请求资源时，系统会进行一次“预演”：它假设批准了这次请求，然后检查系统是否还处于“安全状态”。一个安全状态是指系统存在一个安全的进程执行序列，可以使得所有进程都能最终完成。如果批准请求后系统仍然是安全的，那么就正式分配资源；否则，进程必须等待。

7. 页错误（Page Fault）的处理步骤

当发生页错误时，操作系统的处理流程如下：

1. 硬件陷入（Trap）到操作系统内核，并保存当前进程的状态。
2. 操作系统确定导致错误的虚拟地址，并检查该地址是否合法。如果地址非法，则终止进程。
3. 如果地址合法，操作系统需要从磁盘上加载对应的页。它会查找一个空闲的物理内存页框。
4. 如果没有空闲页框，操作系统会执行页面置换算法（如LRU），选择一个“牺牲”页。如果牺牲页是“脏”的（被修改过），则必须先将其写回磁盘。
5. 操作系统向磁盘发出请求，将所需的页面读入到准备好的物理页框中。
6. 磁盘I/O完成后，操作系统更新进程的页表，将新的映射关系（虚拟页号 -> 物理页号）填入，并设置有效位为1。
7. 恢复进程的运行状态，并重新执行导致错误的指令。

8. 文件系统中的数据与元数据

“数据”和“元数据”是文件系统中的两个核心概念。

- **区别**：“数据”是文件的实际内容，即用户存放的信息，例如文本文档中的文字、图像文件中的像素。而“元数据”是用来描述文件属性的信息，即“关于数据的数据”。
- 例子：以一个名为homework.docx 的文件为例。
  - **数据**：是你在Word中输入的文字、插入的图片等所有实际内容。
  - **元数据**：是存储在文件的索引节点（inode）中的信息，包括文件名、文件大小（如50KB）、所有者、权限（如读写执行权限）、创建时间、最后修改时间，以及指向存储文件内容的磁盘块的指针列表。

9. 资源池管理的同步原语选择

对于管理一个包含3台打印机的共享资源池，最合适的同步原语是**信号量（Semaphore）**。

- 选择与理由：
  1. **初始化**：可以将信号量的初始值设置为3，代表有3台可用的打印机。
  2. **资源获取**：当一个线程需要打印机时，它调用 `sem_wait()`。此操作会使信号量的值减1。如果结果非负，线程获得打印机。如果结果为负（即值为0时调用），线程将被阻塞，表示无可用打印机。
  3. **资源释放**：当线程使用完打印机后，它调用 `sem_post()`，使信号量的值加1，并可能唤醒一个正在等待的线程。
  - 这种方式能够简单而有效地管理N个相同资源的分配，而互斥锁只能管理单个资源（相当于N=1的信号量），条件变量则需要更复杂的代码（需要配合互斥锁和手动计数）来实现相同功能。

10. 策略与机制的区别

“策略”和“机制”是操作系统设计中的两个基本概念，用于分离不同的设计关注点。

- 区别：
  - **机制（Mechanism）**：回答“如何做”的问题。它指的是实现某个功能所需要的基础能力或工具。机制本身不带任何决策性，力求通用和高效。
  - **策略（Policy）**：回答“做什么”或“决定什么”的问题。它指的是利用底层机制来做出决策的算法或规则。
- 以CPU调度为例：
  - **机制**：上下文切换是机制。它提供了保存和恢复进程状态的能力，使得CPU可以在不同进程间切换。
  - **策略**：哪一个进程应该在何时运行，这就是策略。例如，RR（轮转）或SJF（最短作业优先）算法就是不同的调度策略，它们利用上下文切换这个机制来执行自己的决策。

计算题

1. CPU调度算法性能分析

假设有以下5个作业到达系统，请分别计算在使用**抢占式最短作业优先（SJF/STCF）**和**时间片为2的轮转（RR）**这两种调度算法时，所有作业的平均周转时间（Average Turnaround Time）和平均响应时间（Average Response Time）。

作业	到达时间	运行时间
作业A	0	7
作业B	2	4
作业C	3	2
作业D	5	1
作业E	8	5

2. 虚拟地址到物理地址的转换

在一个32位虚拟地址空间的系统中，页面大小为4KB（212 字节）。系统使用一个二级页表进行地址转换，其中页目录项（PDE）和页表项（PTE）的大小均为4字节。一个页目录和一个页表都恰好可以装满一个页面。

请问对于虚拟地址 0x00803456，它对应的：

a) 页目录索引（Page Directory Index）是多少？

b) 页表索引（Page Table Index）是多少？

c) 在页面内的偏移量（Offset）是多少？

3. 磁盘调度寻道成本

一个磁盘的磁道范围为0到199。当前磁头停留在**磁道50**处，并且刚刚完成了对磁道40的访问，正准备向磁道号增加的方向移动。现有如下一个磁盘请求队列（按到达时间排序）：`95, 180, 35, 120, 15, 130, 65, 70`。

请分别计算使用以下两种磁盘调度算法来处理此队列时，磁头的总移动距离（总寻道长度）是多少？

a) SSTF（最短寻道时间优先）

b) C-SCAN（循环扫描）

4. Inode与最大文件大小

假设一个文件系统的块（Block）大小为4KB。每个磁盘块地址需要4个字节来存储。现在，一个inode结构包含以下内容：

- 10个直接指针（Direct Pointers）
- 1个单级间接指针（Single Indirect Pointer）
- 1个双级间接指针（Double Indirect Pointer）

请计算，使用这样一个inode结构，它所能支持的文件的最大大小是多少？

原理阐述题

1. 进程与线程的比较

a) **定义与角色**

- **进程（Process）** 是操作系统进行资源分配和保护的基本单位。你可以把它看作是一个正在运行的程序的实例。它包含了一个程序运行所需的所有资源，并与其他进程隔离。
- **线程（Thread）** 是CPU调度的基本单位，它也被称为轻量级进程。线程是进程内部的一条执行路径。一个进程可以包含一个或多个线程，这些线程协同工作来完成任务。

b) **资源所有权**

- **进程**拥有独立的资源。每个进程都有自己独立的地址空间（代码、数据、堆）、文件描述符表、用户ID以及其他系统资源。这种独立性提供了很好的保护，一个进程的崩溃通常不会影响其他进程。
- **线程**共享其所属进程的大部分资源。同一进程内的所有线程共享该进程的地址空间、全局变量、堆、打开的文件等。但每个线程拥有自己私有的一套资源，主要包括：程序计数器（PC）、寄存器集合和栈。栈的独立性使得每个线程都能独立地执行函数调用。

c) **开销与性能**

- **创建与销毁**：创建进程的开销很大，因为操作系统需要为其分配一整套独立的资源。销毁进程时也需要回收所有这些资源。相比之下，创建线程的开销很小，因为它只需分配一个私有的栈和寄存器集，大部分资源是共享的，因此速度快得多。
- **切换**：进程间的上下文切换开销很大。这不仅需要切换CPU的寄存器状态，还需要切换整个地址空间（例如更新页表基址寄存器），这通常会导致TLB和CPU缓存失效，降低性能。而同一进程内线程间的切换开销则小得多，因为它们共享地址空间，切换时无需改变内存映射，TLB等缓存依然有效，只需切换线程私有的PC、寄存器和栈即可。

d) **通信与同步**

- **进程间通信（IPC）** 相对复杂和缓慢。因为进程间相互隔离，它们的通信必须依赖操作系统提供的机制，例如管道（Pipe）、消息队列（Message Queue）、共享内存（Shared Memory）或套接字（Socket）。
- **线程间通信** 非常简单和快速。因为它们共享内存，一个线程可以直接读写另一个线程的数据。但也正因如此，必须使用同步原语（如互斥锁、信号量）来小心地管理对共享数据的访问，以避免竞争条件和数据不一致的问题。

2. Inode与文件路径解析

当操作系统执行 `cat /home/user/file.txt` 命令时，它通过以下一系列步骤来解析路径并找到文件：

1. **从根目录开始**：内核知道根目录 `/` 的inode号（这通常是一个固定的值，如2）。这是整个查找过程的起点。
2. **读取根目录的inode**：内核读取根目录的inode。这个inode包含了根目录的元数据，最重要的是指向存储根目录内容的磁盘块的指针。
3. **查找“home”**：内核读取根目录的数据块。目录文件是一种特殊的文件，其内容是一系列的<文件名, inode号>映射。内核在这些条目中搜索名为 `home` 的项，并从中获得 `home` 目录的inode号。
4. **读取“home”目录的inode**：现在，内核有了 `home` 目录的inode号，它会去读取这个inode，并找到 `home` 目录的数据块。

5. **查找“user”**：内核在 `home` 目录的数据块内容中搜索名为 `user` 的条目，并获得 `user` 目录的inode号。
6. **读取“user”目录的inode**：同理，内核读取 `user` 目录的inode，找到其数据块。
7. **查找“file.txt”**：内核在 `user` 目录的数据块中搜索名为 `file.txt` 的条目，并最终获得 `file.txt` 这个文件的inode号。
8. **访问文件**：至此，路径解析完成。内核有了 `file.txt` 的inode号。它会读取这个最终的inode，检查文件权限（`cat` 命令需要读权限）。如果权限允许，内核就会根据inode中的指针，找到存储文件实际内容的磁盘块，并将数据读出，最终显示在屏幕上。

在这个过程中，**目录文件**的角色是作为“电话本”，提供了从“名字”到“地址（inode号）”的映射。**Inode**则代表了文件本身，存储了文件的所有属性以及指向文件内容的“地图”。

3. 硬链接与符号链接的实现原理

a) **创建过程**

- **硬链接（Hard Link）**：创建一个硬链接时，文件系统会在指定的目录中创建一个新的目录项，但让这个新条目指向**已存在**的目标文件的inode。它**不会创建新的inode**。同时，它会将那个inode内部的“链接计数（link count）”加1。
- **符号链接（Symbolic Link）**：创建一个符号链接时，文件系统会**创建一个全新的文件和全新的inode**。这个新文件是一个特殊类型的文件，其数据块中存储的内容是它所指向的目标文件的**路径字符串**（例如，“/home/user/file.txt”）。

b) **删除源文件后的影响**

- **硬链接**：删除源文件，实际上只是删除了源文件那个目录项，并将对应inode的链接计数减1。只要这个inode的链接计数还不为0（因为硬链接还存在），inode和其指向的实际数据块就**不会被删除**。文件内容依然可以通过硬链接访问。
- **符号链接**：删除源文件后，符号链接本身依然存在，但它指向的路径已经失效了。此时，这个符号链接就成了一个“悬空链接”或“坏链接”。尝试通过它访问文件将会失败，系统会报告“文件未找到”。

c) **使用限制**

- 硬链接：
  - **不能跨文件系统（分区）**：因为inode号只在它所属的单个文件系统内是唯一的。
  - **通常不能指向目录**：这是为了防止在文件系统中产生循环引用，避免`du`、`find`等工具陷入无限递归。
- 符号链接：
  - **可以跨文件系统**：因为它存储的是一个路径字符串，与inode号无关，所以可以指向任何位置。
  - **可以指向目录**：这是一种非常常见的用法。